PrepPath E-books Store

DBMS

Handwritten Notes

Database Management Systems
made easy for freshers

★ Hand-drawn diagrams for every idea

★ SQL queries, tested & ready to run

★ Real-world logic & interview problems

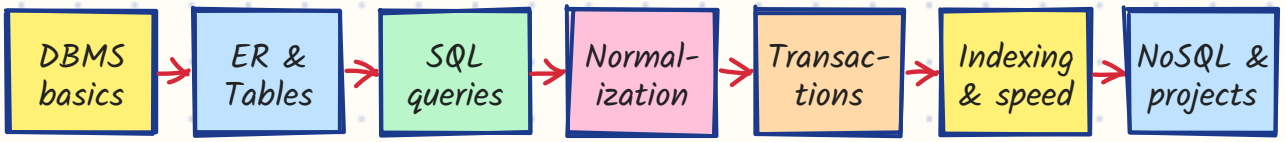
★ 30-day practice plan & web references

Learn it. Draw it. Query it.

START HERE

Contents & Roadmap

These notes explain **DBMS (Database Management Systems)** the way a friend would on a whiteboard: simple words, a picture for every idea, and **SQL you can run**. Follow the order below; every chapter builds on the one before.



Learning order: left to right. SQL is the heart of every interview!

Chapter	What you learn	Page
DBMS Basics	What, why, architecture	3
ER Model	Entities, relationships	5
Relational Model	Tables, keys, algebra	7
SQL Queries	Select, joins, subqueries	11
SQL Practice	Interview-style problems	23
Normalization	FDs, 1NF to BCNF, design	25
Transactions	ACID, locks, recovery	32
Indexing & Storage	B+ tree, hashing, speed	37
Advanced DB	Security, triggers, NoSQL	41
Practice & Guide	Project, Q&A, plan	45

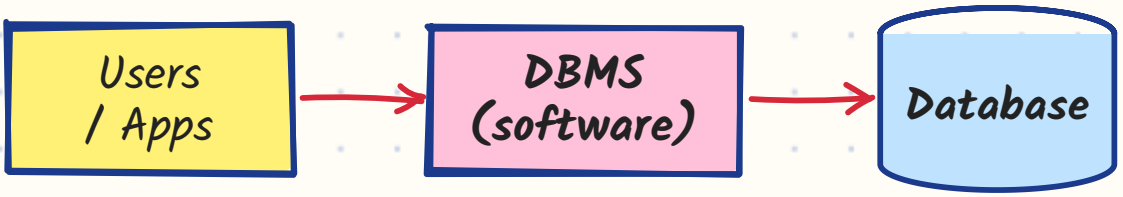
Study trick

How to study each page: 1) read the idea 2) cover the SQL and try writing it 3) run it in SQLite / MySQL 4) solve the practice problems.

BASICS

What Is a Database & DBMS?

Data = raw facts (a name, a price). A **database** = organised collection of data. A **DBMS** is the *software* that stores, protects and retrieves it, like **MySQL, PostgreSQL, Oracle, SQLite, SQL Server**.



You never touch the files directly. The DBMS is the safe gatekeeper.

Why not just use files (Excel / text)?

Problem with files	How a DBMS fixes it
Same data copied everywhere (redundancy)	Store once, link by keys
Two people edit together → mess	Concurrency control, locks
Crash halfway → half-saved data	Transactions & recovery
Anyone can open the file	Users, roles, permissions
Slow search in big files	Indexes, query optimiser
No rules (age = -5 allowed)	Constraints keep data valid

Real-world examples

- **Bank:** accounts and balances (must never be wrong).
- **Amazon:** products, orders, customers, reviews.
- **IRCTC:** seats and bookings at the same second.
- **WhatsApp / Instagram:** users, messages, posts.

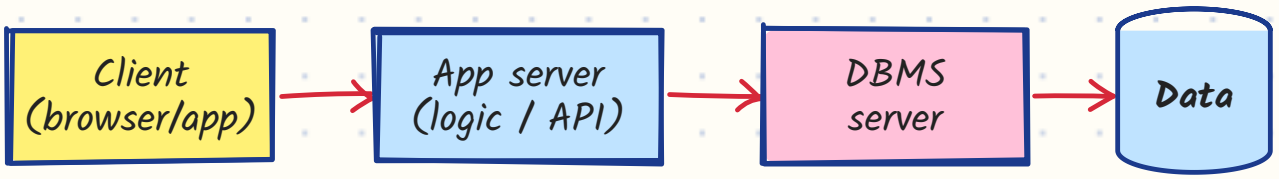
Don't confuse

DBMS vs RDBMS: an RDBMS stores data in **tables (rows & columns)** linked by keys. MySQL & PostgreSQL are RDBMS. Almost every fresher interview means RDBMS + SQL.

DBMS Architecture & Data Independence

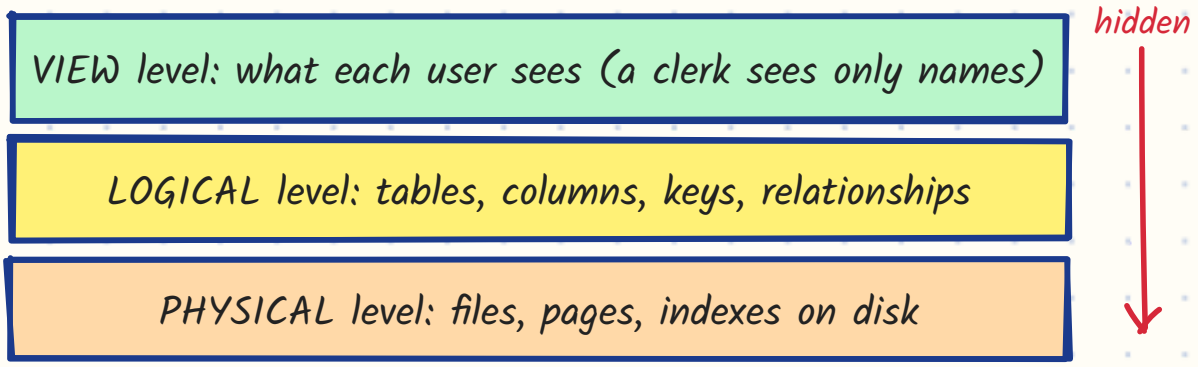
BASICS

3-tier architecture (how apps use a DB)



The user never talks to the database directly.

3 levels of abstraction (3-schema)



Top = simple for users; bottom = complex details are hidden.

Data independence

- **Physical independence:** change how data is stored (new index) → apps still work.
- **Logical independence:** add a column to a table → old views and apps still work.

Language	Meaning	Commands
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE
DML	Change data	INSERT, UPDATE, DELETE
DQL	Query data	SELECT
DCL	Permissions	GRANT, REVOKE
TCL	Transactions	COMMIT, ROLLBACK, SAVEPOINT

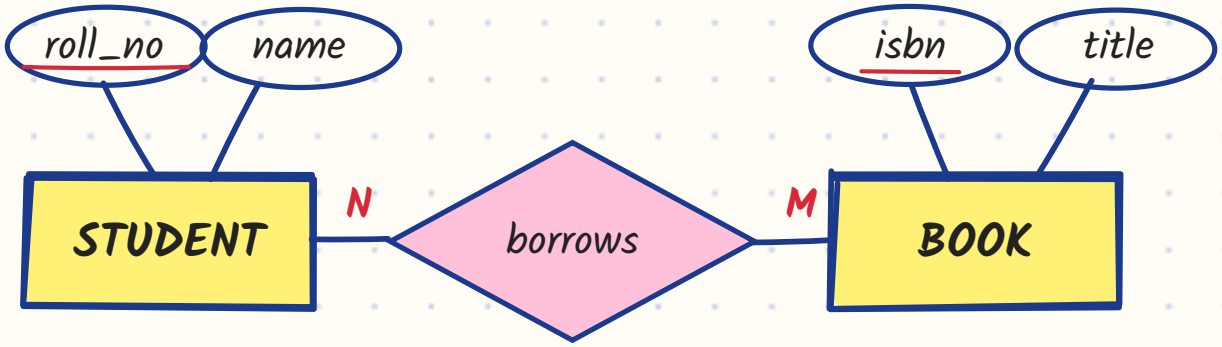
Remember
Mnemonic: **D**efine, **M**anipulate, **Q**uery, **C**ontrol, **T**ransact.

ER Model: Entities & Attributes

ER MODEL

Before building tables, we **draw a picture** of the real world: an **ER (Entity-Relationship) diagram**. It has 3 ingredients.

- **Entity:** a real thing (Student, Book). Drawn as a **rectangle**.
- **Attribute:** a property (name, age). Drawn as an **oval**; the key is underlined.
- **Relationship:** how things connect (Student *borrows* Book). Drawn as a **diamond**.



Underlined attribute = primary key.

Types of attributes

Type	Meaning	Example
Simple	cannot be split	age
Composite	can be split	name → first + last
Multivalued	many values	phone numbers
Key	unique identifier	roll_no

Strong vs weak entity

Strong entity: has its own key (Customer). **Weak** entity: needs its owner (OrderItem needs an Order).

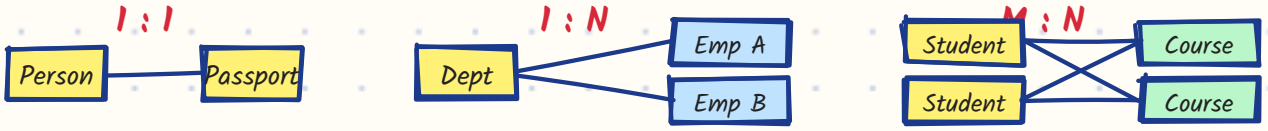
Trick

Interview tip: nouns in the problem statement → entities; adjectives → attributes; verbs → relationships.

ER MODEL

Relationships & Cardinality

Cardinality says *how many* of one entity connect to the other. It decides how you build tables.



One-to-one, one-to-many, many-to-many

Type	Example	How to make tables
1 : 1	Person – Passport	Put one table's key inside the other (or merge)
1 : N	Department – Employee	Put the *'1' side's key* as a foreign key in the *'N' side*
M : N	Student – Course	Make a *third (junction) table* holding both keys

M:N becomes a junction table

```
CREATE TABLE student(sid INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE course(cid INTEGER PRIMARY KEY, title TEXT);
CREATE TABLE enrolment(
  sid INTEGER REFERENCES student(sid),
  cid INTEGER REFERENCES course(cid),
  grade TEXT,
  PRIMARY KEY (sid, cid)       -- composite key
);
INSERT INTO student VALUES (1,'Asha');
INSERT INTO course VALUES (10,'DBMS');
INSERT INTO enrolment VALUES (1,10,'A');
SELECT s.name, c.title, e.grade FROM enrolment e JOIN student s ON
s.sid=e.sid JOIN course c ON c.cid=e.cid;
```

name	title	grade
Asha	DBMS	A

Also know

Participation: *total* = every entity must take part (every Order needs a Customer, a double line). *Partial* = optional (a Customer may have zero orders).

RELATIONAL

The Relational Model: Terms

In the relational model all data lives in **relations**, which are simply **tables**. Learn these words; interviewers love them.

id	name	dept_id	salary
1	Asha	1	60000
2	Bala	1	50000
3	Chitra	2	45000

Table = Relation (this one is called employees)

Formal word	Everyday word	Here
Relation	Table	employees
Tuple	Row / record	(2, Bala, 1, 50000)
Attribute	Column / field	salary
Domain	Allowed values	salary > 0 integers
Degree	Number of columns	4
Cardinality	Number of rows	3
Schema	Blueprint (names + types)	employees(id, name, dept_id, salary)

Properties of a relation

- Every cell holds **one atomic value** (no lists inside a cell).
- Every row is **unique**; row order does not matter.
- Column names are unique; column order does not matter.
- All values in a column have the **same type**.

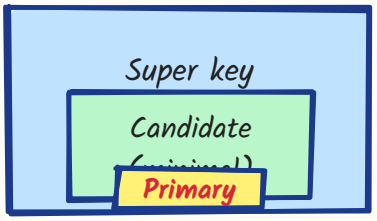
Big idea

NULL means *unknown / missing*. It is **not** 0 and **not** an empty string. `NULL = NULL` is not true; use `IS NULL`.

RELATIONAL

Keys: The Identity System

A **key** is a column (or group) that helps identify rows. Keys are the most asked DBMS topic!



Super key ⊇ Candidate key ⊇ Primary key

Others left over = Alternate keys

Foreign key = links to another table

Think of nested circles: the primary key is the one chosen candidate.

Key	Meaning	Example (student)
Super key	Any combination that is unique	{roll}, {roll,name}, {email}
Candidate key	Smallest unique set	{roll}, {email}
Primary key	Chosen candidate: unique + NOT NULL	roll
Alternate key	Candidates not chosen	email
Composite key	Primary key of 2+ columns	(sid, cid)
Foreign key	Refers to a primary key elsewhere	employees.dept_id → departments.id
Surrogate key	Auto-number with no meaning	id AUTOINCREMENT

Common question

A table has **one** primary key but can have **many** candidate keys and **many** foreign keys. Primary key never NULL; a foreign key **can** be NULL.

foreign key in action

```
SELECT e.name, d.name AS dept FROM employees e JOIN departments d ON e.dept_id = d.id WHERE e.id = 2;
```

name	dept
Bala	IT

RELATIONAL

Integrity Constraints

Constraints are rules the database enforces so bad data can never get in.

Constraint	Rule	Example
NOT NULL	value required	name TEXT NOT NULL
UNIQUE	no duplicates (NULL allowed)	email TEXT UNIQUE
PRIMARY KEY	unique + not null	id INTEGER PRIMARY KEY
FOREIGN KEY	must exist in parent table	REFERENCES departments(id)
CHECK	custom condition	CHECK (salary > 0)
DEFAULT	fill if not given	status DEFAULT 'new'

```
CREATE TABLE account(
  id      INTEGER PRIMARY KEY,
  email   TEXT NOT NULL UNIQUE,
  balance INTEGER DEFAULT 0 CHECK (balance ⩾ 0)
);
INSERT INTO account(email) VALUES ('a@x.com');
SELECT * FROM account;
```

id	email	balance
1	a@x.com	0

Referential integrity: ON DELETE

- **CASCADE:** delete the parent → children deleted too.
- **SET NULL:** children's foreign key becomes NULL.
- **RESTRICT / NO ACTION:** refuse to delete the parent while children exist.

Best practice

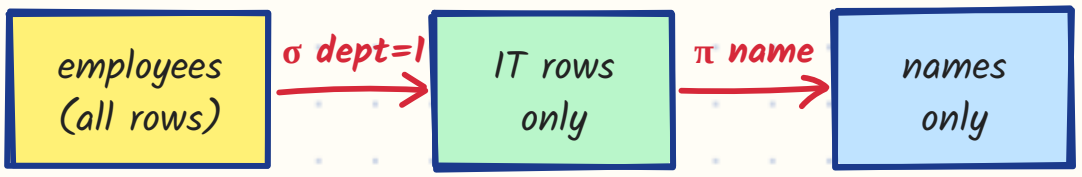
Constraints protect data ***inside the database***, even if the app has a bug. Always use them.

RELATIONAL

Relational Algebra

Relational algebra is the maths behind SQL: a few operators that take tables and return tables. SQL `SELECT` is built from these.

Symbol	Name	Meaning	SQL idea
σ	Select	pick rows by condition	WHERE
π	Project	pick columns	SELECT col
∪	Union	rows in A or B	UNION
−	Difference	in A but not B	EXCEPT
×	Cross product	every pair	CROSS JOIN
⋈	Join	matching pairs	JOIN ... ON
ρ	Rename	rename table/column	AS



Operators chain like pipes: σ then π.

Same thing written in SQL

project + select

```
-- π name ( σ dept_id=1 (employees) )
SELECT name FROM employees WHERE dept_id = 1;
```

name
Asha
Bala
Esha

Union-compatible rule

∪, −, ∩ need both tables to have the **same number of columns with matching types**.

Insight

Joins as algebra: natural join = cross product (×) + select (σ) matching columns + project (π) to drop duplicate columns.

SQL Basics: DDL

SQL (Structured Query Language) talks to every relational database. First we **define** the structure with DDL.

Common data types

Type	Stores	Example
INT / INTEGER	whole numbers	25
DECIMAL(10,2)	exact money	499.50
VARCHAR(n) / TEXT	text	'Asha'
DATE / DATETIME	dates & time	'2026-11-09'
BOOLEAN	true/false	1 / 0

create / alter

```
CREATE TABLE product(
  id    INTEGER PRIMARY KEY,
  name  TEXT NOT NULL,
  price REAL CHECK (price > 0)
);
ALTER TABLE product ADD COLUMN stock INTEGER DEFAULT 0;    -- change
structure
CREATE INDEX idx_name ON product(name);                    -- speed up
search
SELECT name FROM sqlite_master WHERE type='table';
```

name
product

Command	What it does	Can ROLLBACK?
DROP TABLE t	deletes table + structure	no
TRUNCATE TABLE t	deletes all rows fast, keeps table	mostly no
DELETE FROM t	deletes rows (can filter)	yes

Interview favourite

DROP vs TRUNCATE vs DELETE is a classic question. DROP removes the table itself; TRUNCATE empties it; DELETE removes chosen rows and logs each one.

SQL Basics: DML (Insert, Update, Delete)

SQL

DML changes the *data* inside tables. Golden rule: **always write WHERE** with UPDATE and DELETE.

INSERT

```
INSERT INTO departments VALUES (5, 'Legal');
INSERT INTO customers(id,name,city) VALUES (5,'Esha','Salem'),
(6,'Farhan','Salem');
SELECT COUNT(*) FROM customers;
```

COUNT(*)
6

UPDATE (raise for IT)

```
UPDATE employees SET salary = salary + 5000 WHERE dept_id = 1;
SELECT name, salary FROM employees WHERE dept_id = 1 ORDER BY id;
```

name	salary
Asha	65000
Bala	55000
Esha	55000

DELETE small orders

```
DELETE FROM orders WHERE amount < 150;
SELECT COUNT(*) FROM orders;
```

COUNT(*)
4

Danger!

`UPDATE employees SET salary = 0;` with **no WHERE** changes **every** row. Test with a SELECT first, and use a transaction.

Insert from another table

```
CREATE TABLE big_orders AS SELECT * FROM orders WHERE amount ⩾ 300;
SELECT COUNT(*) FROM big_orders;
```

COUNT(*)
3

- `INSERT ... SELECT` copies rows from one table into another.
- In real apps, never build SQL by joining strings (see SQL Injection page).

SELECT & WHERE: Asking Questions

SQL

`SELECT` is the question you ask most. Pattern: ***SELECT*** columns ***FROM*** table ***WHERE*** condition.

rows matching a condition

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

name	salary
Asha	60000
Dev	55000

Operator	Use	Example
= <> < > <= >=	compare	salary >= 50000
AND / OR / NOT	combine	dept_id=1 AND salary>50000
BETWEEN a AND b	range (inclusive)	salary BETWEEN 40000 AND 50000
IN (...)	one of a list	dept_id IN (1,3)
LIKE	pattern	name LIKE 'A%'
IS NULL	missing value	dept_id IS NULL

IN + BETWEEN

```
SELECT name FROM employees WHERE dept_id IN (2,3) AND salary BETWEEN
40000 AND 50000;
```

name
Chitra
Farid

LIKE: % = any text, _ = one char

```
SELECT name FROM employees WHERE name LIKE '%a' ;      -- ends with a
```

name
Asha
Bala
Chitra
Esha
Gita

Classic bug

`WHERE dept_id = NULL` returns ***nothing***. Always use `IS NULL` or `IS NOT NULL`.

ORDER BY, LIMIT, DISTINCT, NULL

SQL

sort + top 3

```
SELECT name, salary FROM employees ORDER BY salary DESC, name ASC
LIMIT 3;
```

name	salary
Asha	60000
Dev	55000
Bala	50000

DISTINCT removes duplicates

```
SELECT DISTINCT city FROM customers ORDER BY city;
```

city
Chennai
Madurai
Trichy

pagination (page 2, size 2)

```
SELECT name FROM employees ORDER BY salary DESC LIMIT 2 OFFSET 2;
-- skip 2, take 2
```

name
Bala
Esha

Working with NULL

`NULL = NULL` is not true, `NULL + 5` is NULL, `COUNT(col)` skips NULLs. `COALESCE(x, 0)` gives the first non-NULL.

replace NULL with a default

```
SELECT name, COALESCE(dept_id, 0) AS dept FROM employees WHERE id =
7;
```

name	dept
Gita	0

AS = give a nickname (alias)

```
SELECT name AS employee, salary * 12 AS yearly FROM employees WHERE
id = 1;
```

employee	yearly
Asha	720000

Memorise

Order of words: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT. You will use this order daily.

SQL Aggregates & GROUP BY

Aggregate functions squeeze many rows into one answer. **GROUP BY** makes one answer *per group*.

Function	Gives	Note
COUNT(*)	number of rows	counts NULL rows too
COUNT(col)	non-NULL values	
SUM / AVG	total / mean	ignore NULLs
MIN / MAX	smallest / largest	work on text & dates too

```
SELECT COUNT(*), SUM(salary), MIN(salary), MAX(salary) FROM
employees;
```

COUNT(*)	SUM(salary)	MIN(salary)	MAX(salary)
7	330000	30000	60000

dept	salary
1	60000
1	50000
2	45000
1	50000

GROUP BY dept: rows with the same dept fall into one bucket, then SUM runs per bucket.

headcount & payroll per department

```
SELECT dept_id, COUNT(*) AS people, SUM(salary) AS payroll
FROM employees GROUP BY dept_id ORDER BY dept_id;
```

dept_id	people	payroll
NULL	1	30000
1	3	160000
2	1	45000
3	2	95000

Rule

Every column in SELECT must be either **inside an aggregate** or **in GROUP BY**. Otherwise the database does not know which value to show.

SQL

HAVING & Order of Execution

`WHERE` filters *rows before* grouping. `HAVING` filters *groups after* grouping.

customers who spent more than 250

```
SELECT customer_id, SUM(amount) AS spent FROM orders
GROUP BY customer_id HAVING SUM(amount) > 250 ORDER BY customer_id;
```

customer_id	spent
1	900
2	700

WHERE first, then HAVING

```
SELECT dept_id, COUNT(*) FROM employees
WHERE salary ⩾ 45000 GROUP BY dept_id HAVING COUNT(*) ⩾ 2 ORDER BY
dept_id;
```

dept_id	COUNT(*)
1	3

The secret order the database really runs



We write SELECT first, but it runs 5th! That is why an alias from SELECT cannot be used in WHERE.

Interview Q

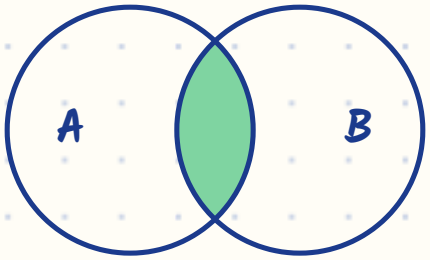
WHERE vs HAVING is asked in almost every interview. WHERE: individual rows, cannot use aggregates. HAVING: groups, can use aggregates.

	WHERE	HAVING
Filters	rows	groups
Runs	before GROUP BY	after GROUP BY
Aggregates allowed	no	yes

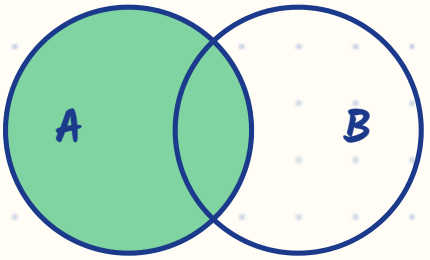
SQL

JOINs (1): INNER & LEFT

Data is split across tables to avoid repetition. A **JOIN** glues rows back together using a matching column (usually primary key = foreign key).



INNER JOIN: only matches



LEFT JOIN: all of A + matches

INNER: Gita has no dept, so she disappears

```
SELECT e.name, d.name AS dept FROM employees e
INNER JOIN departments d ON e.dept_id = d.id ORDER BY e.id;
```

name	dept
Asha	IT
Bala	IT
Chitra	HR
Dev	Sales
Esha	IT
Farid	Sales

LEFT: Admin has 0 staff but still appears

```
SELECT d.name, COUNT(e.id) AS staff FROM departments d
LEFT JOIN employees e ON e.dept_id = d.id GROUP BY d.name ORDER BY
d.name;
```

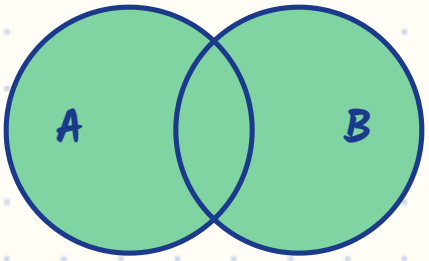
name	staff
Admin	0
HR	1
IT	3
Sales	2

Tip

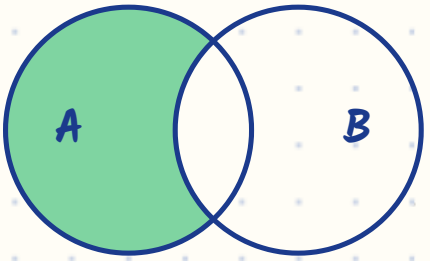
Use **LEFT JOIN** when you must keep every row from the main table even without a match. Use `COUNT(e.id)` not `COUNT(*)` to count real matches.

JOINs (2): RIGHT, FULL, SELF, CROSS

SQL



FULL JOIN: everything from both



LEFT + WHERE B is NULL: only in A

FULL: finds unmatched on both sides

```
SELECT e.name, d.name FROM employees e
FULL JOIN departments d ON e.dept_id = d.id
WHERE e.id IS NULL OR d.id IS NULL ORDER BY e.name;
```

name	name
NULL	Admin
Gita	NULL

SELF JOIN: a table joined with itself

```
SELECT e.name AS employee, m.name AS manager FROM employees e
JOIN employees m ON e.manager_id = m.id ORDER BY e.id;
```

employee	manager
Bala	Asha
Esha	Asha
Farid	Dev

CROSS JOIN: every pair

```
SELECT COUNT(*) FROM customers CROSS JOIN departments;   -- 4 x 4
```

COUNT(*)
16

Join	Keeps
INNER	matching rows only
LEFT / RIGHT	all rows of one side + matches
FULL	all rows of both sides
CROSS	every combination (m × n)
SELF	same table twice, with aliases

MySQL tip

MySQL has no FULL JOIN. Fake it: `LEFT JOIN ... UNION ... RIGHT JOIN ...`.

SQL Subqueries

A ***subquery*** is a query inside another query. Think: *'first answer a small question, then use it in the big one.'*

above-average earners

```
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees) ORDER BY id;
```

name
Asha
Bala
Dev
Esha

IN: matches any value in the list

```
SELECT name FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE amount ⩾ 700);
```

name
Bharathi

EXISTS: is there at least one match?

```
SELECT name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)
ORDER BY id;
```

name
Arun
Bharathi
Charan

correlated: runs once per outer row (top earner per dept)

```
SELECT name FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees WHERE dept_id =
e.dept_id) ORDER BY id;
```

name
Asha
Chitra
Dev

Careful

Prefer ***EXISTS*** for 'is there any?' checks (stops at the first match). ***NOT IN*** breaks if the list contains NULL; use NOT EXISTS instead.

SQL Set Operations & CASE

Operator	Result	Duplicates
UNION	rows in A or B	removed
UNION ALL	A plus B	kept (faster)
INTERSECT	rows in both	removed
EXCEPT / MINUS	in A, not in B	removed

UNION

```
SELECT name FROM customers WHERE city = 'Chennai'
UNION
SELECT name FROM customers WHERE id = 2 ORDER BY 1;
```

name
Arun
Bharathi
Charan

EXCEPT: customers who never ordered

```
SELECT id FROM customers EXCEPT SELECT customer_id FROM orders;
```

id
4

CASE: if / else inside SQL

```
SELECT name, CASE WHEN salary ⩾ 55000 THEN 'High'
                  WHEN salary ⩾ 45000 THEN 'Mid'
                  ELSE 'Low' END AS grade
FROM employees ORDER BY id;
```

name	grade
Asha	High
Bala	Mid
Chitra	Mid
Dev	High

... more rows

Rule

Both queries in a set operation need the **same number of columns** and compatible types.

SQL Window Functions

A **window function** calculates over a group of rows **without collapsing them** (unlike GROUP BY). It is a favourite in interviews.

```
function() OVER (PARTITION BY group_col ORDER BY sort_col)
--              ^ restart per group      ^ order inside group
```

RANK: ties share a rank, then skip

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees ORDER BY rnk, name;
```

name	salary	rnk
Asha	60000	1
Dev	55000	2
Bala	50000	3
Esha	50000	3
Chitra	45000	5

... more rows

Function	On ties 60,50,50,40
ROW_NUMBER()	1, 2, 3, 4 (always unique)
RANK()	1, 2, 2, 4 (gap after tie)
DENSE_RANK()	1, 2, 2, 3 (no gap)

running total

```
SELECT id, amount, SUM(amount) OVER (ORDER BY order_date) AS
running_total
FROM orders ORDER BY id;
```

id	amount	running_total
101	500	500
102	300	800
103	700	1500
104	200	1700
105	100	1800

Next

Also learn **LAG(col)** (previous row) and **LEAD(col)** (next row) for 'compare with yesterday' problems.

SQL Views & CTEs

A **view** is a *saved query* that behaves like a table. A **CTE** (`WITH`) is a *temporary named result* inside one query. Both make big queries readable.

view

```
CREATE VIEW it_staff AS SELECT name, salary FROM employees WHERE
dept_id = 1;
SELECT COUNT(*) FROM it_staff;
```

COUNT(*)
3

View	Why use it
Security	show only some columns/rows
Simplicity	hide a complex JOIN
Reuse	write once, query many times

CTE: step 1 = spend per customer, step 2 = add names

```
WITH spend AS (
  SELECT customer_id, SUM(amount) AS total FROM orders GROUP BY
customer_id
)
SELECT c.name, s.total FROM spend s JOIN customers c ON c.id =
s.customer_id
ORDER BY s.total DESC;
```

name	total
Arun	900
Bharathi	700
Charan	200

Know the difference

A normal view stores **no data**, only the query. A **materialized view** stores the result (faster, but must be refreshed).

SQL PRO SQL Problems (1): Classics

These appear in almost every placement test. Read the question, say the plan, then the SQL.

1. Second highest salary

plan: sort unique salaries, skip the first

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1
OFFSET 1;
```

salary
55000

2. Find duplicate values

plan: group, keep groups bigger than 1

```
SELECT salary, COUNT(*) AS times FROM employees GROUP BY salary
HAVING COUNT(*) > 1;
```

salary	times
50000	2

3. Department with the highest payroll

```
SELECT d.name, SUM(e.salary) AS total FROM employees e JOIN
departments d ON d.id = e.dept_id
GROUP BY d.name ORDER BY total DESC LIMIT 1;
```

name	total
IT	160000

4. Who is a manager?

plan: managers = ids that appear in manager_id

```
SELECT name FROM employees WHERE id IN (SELECT manager_id FROM
employees) ORDER BY id;
```

name
Asha
Dev

Method

Say it in English first: *'group by what? aggregate what? filter before or after?'*

SQL PRO

SQL Problems (2): Real-World

5. Customers who never ordered

LEFT JOIN + IS NULL: the classic anti-join

```
SELECT c.name FROM customers c LEFT JOIN orders o ON o.customer_id =
c.id WHERE o.id IS NULL;
```

name
Divya

6. Highest paid employee in each department

Top-N per group = ROW_NUMBER + PARTITION BY

```
WITH r AS (SELECT name, dept_id, ROW_NUMBER() OVER (PARTITION BY
dept_id ORDER BY salary DESC, id) AS rn
           FROM employees WHERE dept_id IS NOT NULL)
SELECT name, dept_id FROM r WHERE rn = 1 ORDER BY dept_id;
```

name	dept_id
Asha	1
Chitra	2
Dev	3

7. Monthly revenue report

```
SELECT substr(order_date,1,7) AS month, SUM(amount) AS revenue
FROM orders GROUP BY month ORDER BY month;
```

month	revenue
2026-01	800
2026-02	900
2026-03	100

8. Average order value per city

```
SELECT c.city, AVG(o.amount) AS avg_order FROM customers c JOIN
orders o ON o.customer_id = c.id
GROUP BY c.city ORDER BY c.city;
```

city	avg_order
Chennai	275.0
Madurai	700.0

Handwriting helps

Practice habit: write each query **twice** on paper without looking. Then run it. Speed comes from repetition.

DESIGN

Functional Dependencies

A ***functional dependency (FD)*** `X → Y` means: *if you know X, you can find exactly one Y.* Example: `roll_no → name` (a roll number gives one name). The reverse is not true.

roll_no	name	dept	hod
1	Asha	IT	Dr.K
2	Bala	IT	Dr.K
3	Chitra	HR	Dr.M

roll_no → name, dept. dept → hod. Same dept always gives the same hod.

Armstrong rules: reflexive (X → part of X), augmentation (a → b gives ac → bc), transitivity (a → b, b → c gives a → c).

Closure: everything X can determine

Closure X^+ = all attributes you can reach starting from X. If X^+ contains ***all*** attributes, X is a ***super key***; if also minimal, a ***candidate key***.

```
from itertools import combinations
def closure(attrs, fds):
    res = set(attrs)
    changed = True
    while changed:
        changed = False
        for lhs, rhs in fds:
            if set(lhs) ⩽ res and not set(rhs) ⩽ res:
                res ⊨ set(rhs); changed = True
    return res
def candidate_keys(attrs, fds):
    keys = []
    for n in range(1, len(attrs) + 1):
        for c in combinations(attrs, n):
            if closure(c, fds) == set(attrs) and not any(set(k) ⩽
set(c) for k in keys):
                keys.append(c)
    return keys

fds = [("A", "B"), ("B", "C")]
print(closure("A", fds))          # {'A', 'B', 'C'}
```

DESIGN

Anomalies & First Normal Form

Normalization = organising tables so data is stored **once**, so updates cannot create contradictions. A bad single table causes 3 *anomalies*.

order	customer	phone	item
1	Arun	9111	Pen
2	Arun	9111	Book
3	Arun	9999	Bag

Arun's phone is repeated. Row 3 disagrees. Which is right?

Anomaly	Meaning	Example
Update	change in many places, miss one	change phone in 1 of 3 rows
Insert	cannot add data without other data	cannot add a new customer until they order
Delete	deleting loses unrelated facts	delete the only order, lose the customer

1NF: every cell is one atomic value

id	name	phones
1	Asha	111, 222
2	Bala	333

NOT 1NF: two numbers in one cell

id	name	phone
1	Asha	111
1	Asha	222
2	Bala	333

1NF: one value per cell

- No repeating groups or lists inside a cell.
- Each row is unique (has a primary key).
- Better still: put phones in a **separate table** linked by id.

DESIGN

2NF & 3NF

2NF: no partial dependency

Table must be in 1NF **and** no non-key column depends on **only part** of a composite key.

student_id	course_id	student_name	grade
1	C1	Asha	A
1	C2	Asha	B

Key = (student_id, course_id). But student_name depends on student_id alone: partial dependency!

fix: Asha's name stored once

```
CREATE TABLE student(student_id INTEGER PRIMARY KEY, student_name
TEXT);
CREATE TABLE enrol(student_id INT, course_id TEXT, grade TEXT,
PRIMARY KEY(student_id, course_id));
INSERT INTO student VALUES (1,'Asha');
INSERT INTO enrol VALUES (1,'C1','A'),(1,'C2','B');
SELECT COUNT(*) FROM student;
```

COUNT(*)
1

3NF: no transitive dependency

1NF + 2NF and **no non-key column depends on another non-key column**. Bad chain: `emp_id → dept_id → dept_name`.



transitive: emp_id reaches dept_name through dept_id

Fix: move dept_id → dept_name into its own departments table.

Famous line

Memory line: every non-key attribute must depend on **the key, the whole key, and nothing but the key** (so help me Codd).

DESIGN

BCNF & Normal Forms Summary

BCNF (Boyce-Codd NF) is a stricter 3NF: for **every** FD `X → Y`, X must be a **super key**.

student	subject	teacher
Asha	DBMS	Ravi
Bala	DBMS	Ravi
Asha	OS	Sita

Each teacher teaches ONE subject: teacher → subject. But teacher is not a key, so it breaks BCNF.

- **Fix:** split into (teacher, subject) and (student, teacher). Now every determinant is a key.

Form	Rule in one line	Removes
1NF	atomic cells, no repeating groups	lists in cells
2NF	1NF + no partial dependency	part-key dependence
3NF	2NF + no transitive dependency	non-key → non-key
BCNF	every determinant is a super key	remaining odd cases
4NF	no multi-valued dependency	independent multi lists

Decomposition must be 'lossless'

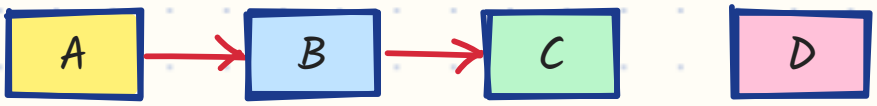
After splitting a table you must be able to **JOIN it back** and get exactly the original rows, nothing lost, nothing extra. Also try to **preserve dependencies**.

> *Practical*
> In most real projects, **3NF** is the target. Go further only when needed.

DESIGN

Finding Keys with Code

Exam problem: *'Given R(A,B,C,D) with A → B, B → C, find all candidate keys.'* Steps: find closures, keep the ***smallest*** attribute set whose closure is everything.



body points to D, so D must be in every key.

Attributes never on a right side must be in the key.

```
from itertools import combinations
def closure(attrs, fds):
    res = set(attrs)
    changed = True
    while changed:
        changed = False
        for lhs, rhs in fds:
            if set(lhs) ⩽ res and not set(rhs) ⩽ res:
                res ⊨ set(rhs); changed = True
    return res
def candidate_keys(attrs, fds):
    keys = []
    for n in range(1, len(attrs) + 1):
        for c in combinations(attrs, n):
            if closure(c, fds) == set(attrs) and not any(set(k) ⩽
set(c) for k in keys):
                keys.append(c)
    return keys

attrs = "ABCD"
fds = [("A", "B"), ("B", "C")]
print(closure("A", fds))              # {A,B,C}   no D yet
print(closure("AD", fds))             # {A,B,C,D} = everything!
print(candidate_keys(attrs, fds))     # [('A', 'D')]
```

Vocabulary

Prime attribute = part of any candidate key. ***Non-prime*** = not in any key. 2NF/3NF rules talk about non-prime attributes.

Denormalization & Schema Design Steps

DESIGN

Design recipe (use this in interviews)

1. List **entities** (nouns): User, Product, Order.
2. List **attributes** and pick a **primary key** for each.
3. Draw **relationships** and their cardinality (1:N, M:N).
4. Convert to **tables** ; add foreign keys; M:N → junction table.
5. Normalise to **3NF** ; add constraints and indexes.



E-commerce: order–product is M:N, so we need an order_item table.

bill = sum(price × qty)

```
CREATE TABLE customer(id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE product(id INTEGER PRIMARY KEY, title TEXT, price INT);
CREATE TABLE ord(id INTEGER PRIMARY KEY, customer_id INT REFERENCES
customer(id));
CREATE TABLE order_item(order_id INT REFERENCES ord(id), product_id
INT REFERENCES product(id), qty INT, PRIMARY KEY(order_id,
product_id));
INSERT INTO customer VALUES (1,'Arun'); INSERT INTO product VALUES
(1,'Pen',10),(2,'Book',50);
INSERT INTO ord VALUES (1,1); INSERT INTO order_item VALUES (1,1,3),
(1,2,1);
SELECT SUM(p.price*i.qty) FROM order_item i JOIN product p ON
p.id=i.product_id WHERE i.order_id=1;
```

SUM(p.price*i.qty)
80

Trade-off

Denormalization = deliberately adding redundancy (e.g. storing total in the order) to make reads **faster**. Used in reports/analytics; cost: harder updates.

Design Practice: Library, Hospital, Cab

DESIGN

Practise turning words into tables. For each: entities, keys, relationships.

1. Library

```
member(member_id PK, name, phone)
book(book_id PK, title, author, copies)
loan(loan_id PK, member_id FK, book_id FK, issue_date, return_date)
-- member 1:N loan, book 1:N loan   ⇒  member M:N book through loan
```

2. Hospital

```
patient(patient_id PK, name, dob)
doctor(doctor_id PK, name, speciality)
appointment(appt_id PK, patient_id FK, doctor_id FK, slot_time,
UNIQUE(doctor_id, slot_time))
-- UNIQUE stops double-booking a doctor at the same time
```

3. Cab booking app

```
rider(rider_id PK, name)      driver(driver_id PK, name, vehicle_no
UNIQUE)
trip(trip_id PK, rider_id FK, driver_id FK, pickup, drop, fare,
status)
payment(payment_id PK, trip_id FK UNIQUE, amount, mode)
-- trip 1:1 payment
```

Thinking tool

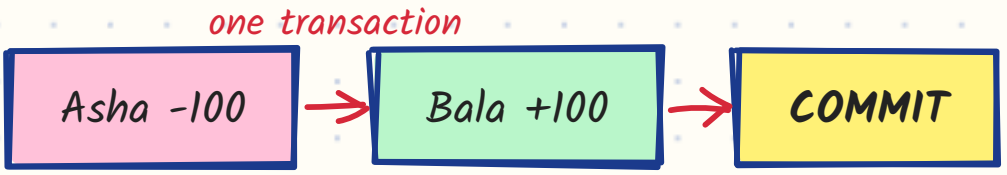
Ask yourself for each table: ***What uniquely identifies a row? What must never be NULL? What must never repeat?*** The answers are your constraints.

- Add ***created_at*** columns to log when rows are made.
- Prefer ***soft delete*** (`is_active = 0`) for important data like users.

TRANSACTIONS

Transactions & ACID

A **transaction** is a group of SQL steps treated as **one unit of work: all happen, or none**. Classic example: transferring ₹100 from Asha to Bala.



If anything fails between the two steps → ROLLBACK (undo both).

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE acc(name TEXT, bal INT CHECK(bal ⩾ 0))")
db.executemany("INSERT INTO acc VALUES (?, ?)", [("Asha", 100),
("Bala", 50)])
db.commit()
try:
    db.execute("UPDATE acc SET bal = bal - 300 WHERE name='Asha'")  #
overdraft!
    db.execute("UPDATE acc SET bal = bal + 300 WHERE name='Bala'")
    db.commit()
except sqlite3.IntegrityError:
    db.rollback()                          # undo everything
print(db.execute("SELECT * FROM acc").fetchall())   # [('Asha',100),
('Bala',50)]
```

ACID: the 4 promises

	Name	Meaning	Bank example
A	Atomicity	all or nothing	no half transfer
C	Consistency	rules stay true	no negative balance
I	Isolation	parallel users don't disturb	two transfers don't mix
D	Durability	survives a crash after COMMIT	power cut can't undo it

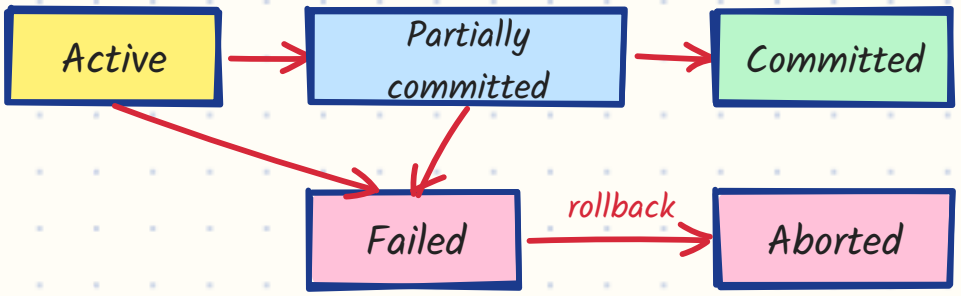
Interview gold

Who guarantees what? Atomicity & Durability → recovery manager (logs). Isolation → concurrency control (locks). Consistency → constraints + correct app logic.

TRANSACTIONS

Transaction States & Schedules

Life cycle of a transaction



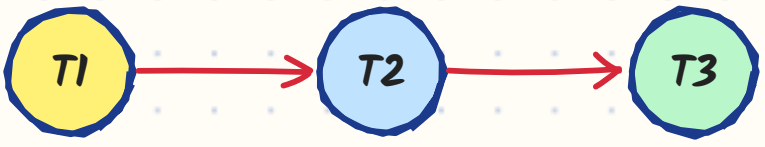
Active → Partially committed → Committed. Any error → Failed → Aborted.

Schedules: interleaved steps of many transactions

Time	T1 (Asha → Bala)	T2 (total report)
1	read A; A = A − 100; write A	
2		read A, read B (A reduced, B not yet!)
3	read B; B = B + 100; write B	

T2 saw an **inconsistent** total. A **serial** schedule (one finishes, then the next) is always safe but slow. We want interleaving that **behaves like serial**: that is **serializability**.

Precedence-graph test



Arrow Ti → Tj: Ti's conflicting step came first. No cycle ⇒ serializable.

Definition
Conflict = two operations on the same item from different transactions where **at least one is a write**.

TRANSACTIONS

Concurrency Problems & Isolation Levels

When many users work at once, these bugs can appear unless the DBMS isolates them:

Problem	What happens	Story
Dirty read	read data another txn has not committed	balance later rolled back
Non-repeatable read	same row read twice gives different values	price changed between reads
Phantom read	same query returns new rows the 2nd time	new order appears mid-report
Lost update	one write overwrites another	two edits of one seat count

Isolation levels (SQL standard)

Level	Dirty	Non-repeatable	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	no	possible	possible
Repeatable Read	no	no	possible*
Serializable	no	no	no

Higher isolation = **safer but slower**. Defaults: PostgreSQL Read Committed, MySQL Repeatable Read.

BEGIN … ROLLBACK leaves data untouched

```
BEGIN;
UPDATE employees SET salary = salary + 1000 WHERE id = 2;
ROLLBACK;                              -- nothing was saved
SELECT salary FROM employees WHERE id = 2;
```

salary
50000

Think

Pick the **lowest level that is still safe**. Money = high; a likes counter = relaxed.

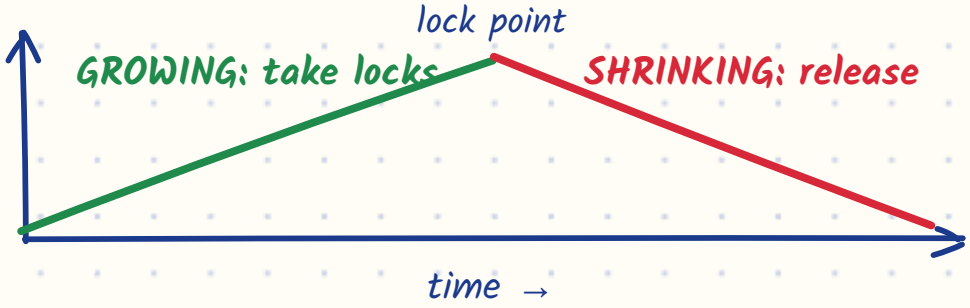
TRANSACTIONS

Locks, 2PL & Deadlock

To keep isolation, the DBMS **locks** data. **Shared (S) lock** = many readers allowed. **Exclusive (X) lock** = one writer, nobody else.

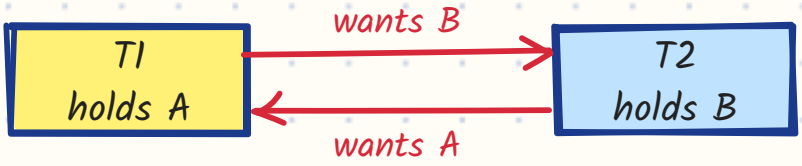
Holding \ Asking	S (read)	X (write)
S	✔ OK	✘ wait
X	✘ wait	✘ wait

Two-Phase Locking (2PL)



Rule: once you release any lock you can never take a new one. This guarantees serializability.

Deadlock



Each waits for the other forever: a cycle!

- **Detect:** build a wait-for graph; a cycle means deadlock → abort one (the *victim*).
- **Prevent:** always lock items in the same order; keep transactions short.

TRANSACTIONS

Recovery: Logs & Checkpoints

Disks and power can fail. The DBMS survives using a **log**, a diary of every change, written **before** the data itself. This is **Write-Ahead Logging (WAL)**.

Log record	Meaning
<T1, start>	transaction began
<T1, A, 500, 400>	T1 changed A: old value 500, new value 400
<T1, commit>	T1 is finished and safe



Log first = safe even if the data pages never reached disk.

After a crash

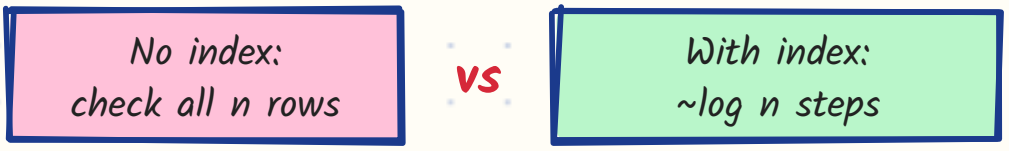
- **REDO** transactions that have <commit> in the log (re-apply their new values).
- **UNDO** transactions with no commit (put the old values back).
- A **checkpoint** marks 'everything before here is safely on disk', so recovery starts from there, not from the beginning.

Link it
Atomicity comes from UNDO; **Durability** comes from REDO. Two sentences that win viva questions.

INDEXING

Indexing: Finding Data Fast

An **index** is like the index at the back of a book: instead of reading every page (full table scan) you jump straight to the right one.



A table of 10 million rows: 10 million checks vs about 24.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, email TEXT)")
db.executemany("INSERT INTO users(email) VALUES (?)",
[(f"u{i}@x.com",) for i in range(1000)])
q = "EXPLAIN QUERY PLAN SELECT * FROM users WHERE email =
'u500@x.com'"
print(db.execute(q).fetchall()[0][3])       # SCAN users
(slow)
db.execute("CREATE INDEX idx_email ON users(email)")
print(db.execute(q).fetchall()[0][3])       # SEARCH users USING
INDEX idx_email
```

Index pros	Index cons
much faster SELECT / WHERE / JOIN / ORDER BY	uses extra disk space
enforces UNIQUE quickly	slows INSERT / UPDATE / DELETE

When to add an index

- Columns used often in **WHERE, JOIN, ORDER BY**.
- Columns with **many different values** (email) are great; yes/no columns are poor.
- Primary keys are indexed **automatically**.

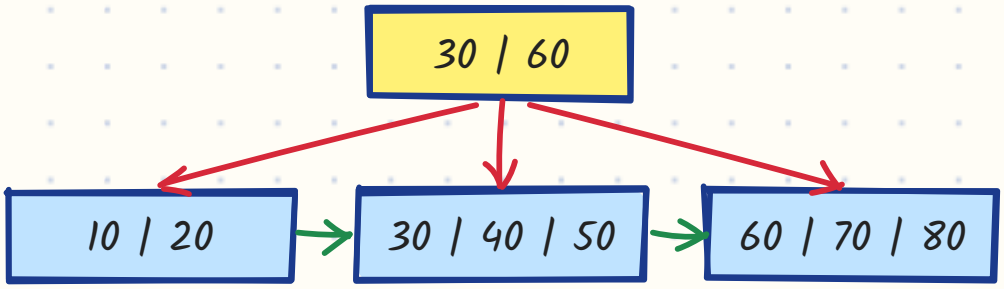
Index killer

`WHERE name LIKE '%an'` (wildcard at the **start**) and `WHERE UPPER(name)= ...` **cannot use** a normal index.

INDEXING

B-Tree & B+ Tree

Most databases store indexes as a **B+ tree**: a short, wide, balanced tree. All data pointers live in the **leaves**, and leaves are **linked** for fast range scans.



green arrows: linked leaves (for range queries)

Search 50: root (30≤50<60) → middle leaf → found. Only 2 hops!

	B-Tree	B+ Tree
Data stored in	all nodes	leaf nodes only
Leaves linked?	no	yes
Range query	slower	fast (follow links)
Used by	some systems	MySQL, PostgreSQL, Oracle

Why so fast?

- Each node = one **disk page**, holding hundreds of keys, so the tree is only **3–4 levels** deep even for millions of rows.
- Always **balanced**: every search costs the same.
- Search, insert and delete are all **O(log n)**.

Connect the dots

Compare with **DSA**: a B+ tree is a BST that got 'wide' so that one node fits one disk read.

Index Types & Query Optimisation

INDEXING

Type	Idea	Note
Clustered	table rows are ***physically sorted*** by this key	only 1 per table (InnoDB primary key)
Non-clustered	separate structure with pointers to rows	many allowed
Composite	index on 2+ columns (a, b)	order matters: leftmost prefix rule
Unique	no duplicate values	enforces constraints
Hash	exact match = only	very fast for =, no ranges

Leftmost prefix rule

```
CREATE INDEX idx_city_name ON customers(city, name);
-- uses index:    WHERE city = 'Chennai'
-- uses index:    WHERE city = 'Chennai' AND name = 'Arun'
-- NOT useful:    WHERE name = 'Arun'          (skips the first column)
```

Slow query? Check the plan

read the plan: SEARCH = good, SCAN = full scan

```
CREATE INDEX idx_city ON customers(city);
EXPLAIN QUERY PLAN SELECT * FROM customers WHERE city = 'Chennai';
```

- Select only needed columns (avoid `SELECT *`).
- Filter early with WHERE; avoid functions on indexed columns.
- Use ***LIMIT*** for big result sets; ***EXPLAIN*** to see the plan.
- Fix the ***N+1 problem***: one JOIN instead of 100 small queries from app code.

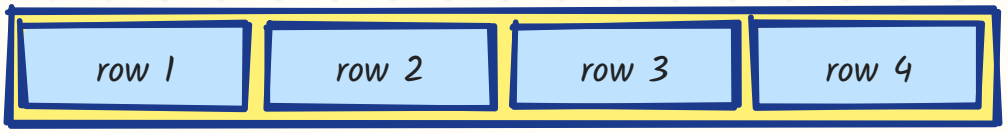
Behind the scenes

The ***query optimiser*** picks the cheapest plan using statistics, so keep stats updated (ANALYZE).

INDEXING

Storage: Pages, Buffer & Row vs Column

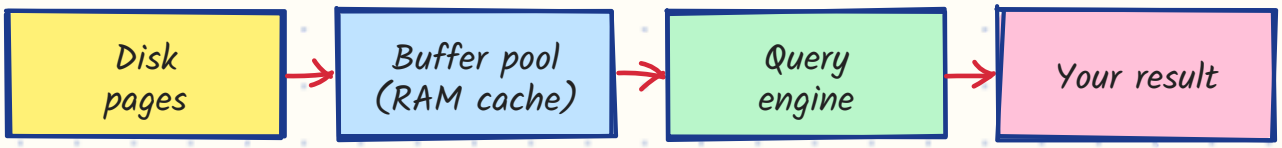
Databases do not read single rows from disk. They read fixed-size **pages (blocks)**, typically 4–16 KB, each holding many rows.



one PAGE = many rows (read together)

Fewer pages read = faster query. Indexes help skip pages.

From disk to your query



Frequently used pages stay in RAM (the buffer pool).

Storage idea	Meaning	Good for
Row store	whole rows together	apps: insert / update (OLTP)
Column store	each column together	analytics, SUM over millions (OLAP)
Heap file	unordered rows	fast inserts
Sorted / indexed file	ordered by key	fast lookups

- **OLTP** = many tiny transactions (banking, shopping). **OLAP** = few heavy reports (data warehouse).
- **RAID / replication** protect against disk failure.

Big picture

Speed trick everywhere in computing: **keep the hot data in memory, minimise disk reads.**

Security: GRANT & SQL Injection

ADVANCED

DCL: who may do what?

```
CREATE USER 'intern'@'localhost' IDENTIFIED BY '...';
GRANT SELECT ON shop.* TO 'intern'@'localhost';   -- read only
GRANT INSERT, UPDATE ON shop.orders TO 'clerk'@'localhost';
REVOKE UPDATE ON shop.orders FROM 'clerk'@'localhost';
```

- ***Least privilege:*** give each user only what they need.
- Never connect your app as the admin / root user.
- Store ***password hashes*** (bcrypt), never plain passwords.

SQL Injection: the #1 web attack

If you paste user text into SQL, an attacker can ***change the query***. Typing `admin' --` as the name turns the rest of the query into a comment, so the password check disappears!

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users(name TEXT, pw TEXT)")
db.execute("INSERT INTO users VALUES ('admin', 'secret')")

def login_bad(u, p):          # DANGEROUS: string pasted into SQL
    q = f"SELECT * FROM users WHERE name='{u}' AND pw='{p}'"
    return db.execute(q).fetchone()

def login_good(u, p):         # SAFE: placeholders (?)
    q = "SELECT * FROM users WHERE name=? AND pw=?"
    return db.execute(q, (u, p)).fetchone()

print(login_bad("admin' --", "wrong"))    # ('admin','secret')
hacked!
print(login_good("admin' --", "wrong"))   # None
```

The fix

Always use parameterised queries / prepared statements. The database treats the values as data, never as SQL code.

Triggers, Procedures & Functions

ADVANCED

Sometimes you want the database itself to run code. Three tools:

Tool	Runs when	Use
Stored procedure	you CALL it	bundle many steps (transfer money)
Function	used inside a query	return a value (tax of price)
Trigger	an INSERT / UPDATE / DELETE happens	auto-audit, auto-update

A trigger that writes an audit log

NEW = the updated row, OLD = the row before

```
CREATE TABLE audit(msg TEXT);
CREATE TRIGGER log_salary AFTER UPDATE OF salary ON employees
BEGIN
  INSERT INTO audit VALUES ('salary changed for ' || NEW.name);
END;
UPDATE employees SET salary = 61000 WHERE id = 1;
SELECT msg FROM audit;
```

msg
salary changed for Asha

Stored procedure (MySQL style)

```
DELIMITER //
CREATE PROCEDURE raise_salary(IN emp INT, IN pct INT)
BEGIN
  UPDATE employees SET salary = salary + salary * pct / 100 WHERE id
= emp;
END //
DELIMITER ;
CALL raise_salary(2, 10);
```

Balance

Pros: faster (runs inside the server), reusable, secure. **Cons:** business logic hidden in the DB, harder to test and version. Use triggers sparingly.

ADVANCED

NoSQL: Beyond Tables

NoSQL ('not only SQL') databases drop the rigid table model to handle ***huge, fast-changing or unstructured data***. They scale out across many cheap servers.

Type	Stores	Examples	Use case
Document	JSON-like documents	MongoDB, Couchbase	product catalogs, profiles
Key-Value	key → value	Redis, DynamoDB	cache, sessions, carts
Column family	wide columns	Cassandra, HBase	time series, logs
Graph	nodes + edges	Neo4j	friends, recommendations

Same data, two worlds

relational

```
-- SQL: 2 tables + JOIN
users(id, name)
orders(id, user_id, item)
```

document

```
// MongoDB: one document
{ "name": "Arun",
  "orders": [ {"item":"Pen"},
              {"item":"Book"} ]
}
```

```
import json
doc = {"name": "Arun", "orders": [{"item": "Pen"}, {"item": "Book"}]}
print(json.dumps(doc))                          # documents are JSON
print([o["item"] for o in doc["orders"]])       # ['Pen', 'Book']
```

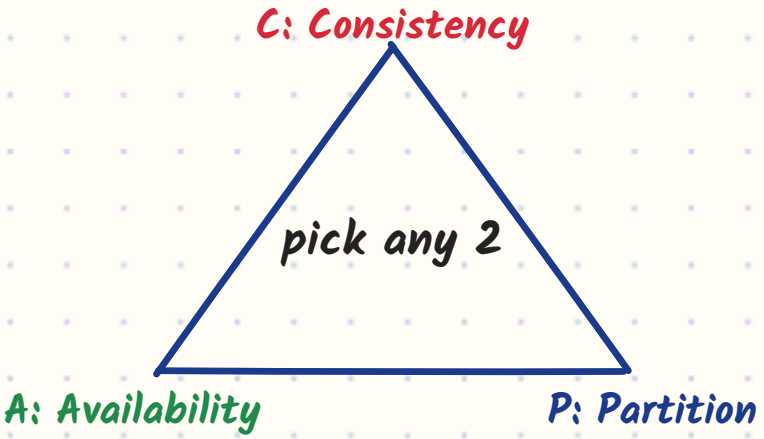
Reality

NoSQL is ***not*** a replacement. Banks still use SQL for strict transactions; many big apps use ***both*** (polyglot persistence).

ADVANCED

CAP, Replication & Sharding

CAP theorem



In a distributed system, when the network splits (P) you must choose C or A.

Choice	Behaviour	Example
CP	stay correct, may refuse requests	MongoDB (default), HBase
AP	always answer, data may be briefly stale	Cassandra, DynamoDB

Scaling a database

Technique	Idea	Helps with
Vertical scaling	bigger machine	simple, has a limit
Replication	copies on several servers (1 leader, many followers)	read speed, backup
Sharding	split rows across servers (e.g. user_id % 4)	huge data & writes
Caching (Redis)	keep hot answers in memory	repeat reads

Interview line

SQL vs NoSQL in one line: SQL = strong consistency, joins, fixed schema. NoSQL = flexible schema, easy horizontal scaling.

- **BASE** (NoSQL style): Basically Available, Soft state, Eventually consistent.
- **ACID** (SQL style): see the transactions chapter.

Using SQL from Python (Real Apps)

PRACTICE

Real programs talk to a DB through a ***driver***: `sqlite3` (built in), `mysql-connector`, `psycopg2`, or an ORM like SQLAlchemy / Django. The pattern is always: ***connect*** → ***execute*** → ***fetch*** → ***commit*** → ***close***.

```
import sqlite3
db = sqlite3.connect(":memory:")
db.row_factory = sqlite3.Row                # access columns by name
db.execute("CREATE TABLE todo(id INTEGER PRIMARY KEY, task TEXT, done
INT DEFAULT 0)")

def add(task):                              # Create
    with db:                                # auto-commit / rollback
        return db.execute("INSERT INTO todo(task) VALUES (?)",
(task,)).lastrowid
def finish(i):                              # Update
    with db: db.execute("UPDATE todo SET done = 1 WHERE id = ?",
(i,))
def pending():                              # Read
    return [r["task"] for r in db.execute("SELECT task FROM todo
WHERE done = 0")]
def remove(i):                              # Delete
    with db: db.execute("DELETE FROM todo WHERE id = ?", (i,))

a = add("learn joins"); b = add("practice window functions")
finish(a)
print(pending())                            # ['practice window
functions']
```

- ***CRUD*** = Create, Read, Update, Delete = INSERT, SELECT, UPDATE, DELETE.
- `with db:` commits on success, rolls back on error.

> *Tip*
> An ***ORM*** maps tables ↔ classes (a row becomes an object). Handy, but learn raw SQL first: you need it to debug slow queries.

Mini Project: Food Ordering Database

PRACTICE

Small food-delivery database. Relationships: restaurant 1:N dish, dish 1:N orders.

schema (3 tables, 2 foreign keys)

```
CREATE TABLE restaurant(id INTEGER PRIMARY KEY, name TEXT, city
TEXT);
CREATE TABLE dish(id INTEGER PRIMARY KEY, restaurant_id INT
REFERENCES restaurant(id), name TEXT, price INT);
CREATE TABLE orders(id INTEGER PRIMARY KEY, customer TEXT, dish_id
INT REFERENCES dish(id), qty INT, order_date TEXT);
```

Q1 revenue per restaurant (3-table join)

```
SELECT r.name, SUM(d.price * o.qty) AS revenue FROM orders o
JOIN dish d ON d.id = o.dish_id JOIN restaurant r ON r.id =
d.restaurant_id
GROUP BY r.name ORDER BY revenue DESC;
```

name	revenue
Spice Hub	780
Curry Leaf	320

Q2 best-selling dish

```
SELECT d.name, SUM(o.qty) AS sold FROM orders o JOIN dish d ON d.id =
o.dish_id
GROUP BY d.name ORDER BY sold DESC LIMIT 1;
```

name	sold
Parotta	4

Q3 repeat customers

```
SELECT customer, COUNT(*) AS orders FROM orders GROUP BY customer
HAVING COUNT(*) > 1 ORDER BY customer;
```

customer	orders
Arun	2
Bala	2

Q4 latest order of each customer

```
SELECT customer, order_date FROM (
  SELECT customer, order_date, ROW_NUMBER() OVER (PARTITION BY
customer ORDER BY order_date DESC) AS rn FROM orders)
WHERE rn = 1 ORDER BY customer;
```

customer	order_date
Arun	2026-11-02
Bala	2026-11-03
Chitra	2026-11-03

PRACTICE

Cheat Sheet: See X, Use Y

Most DBMS and SQL problems are known patterns. Match the clue to the tool.

If you see...	Use this	Page
'for each department / customer ...'	GROUP BY + aggregate	15
'only groups with more than ...'	HAVING	16
'combine data from 2 tables'	JOIN on the key	17
'who has no orders / not present'	LEFT JOIN ... IS NULL, or NOT EXISTS	17, 19
'employee and manager'	SELF JOIN	18
'above average / the maximum'	subquery	19
'Nth highest / top N per group'	LIMIT OFFSET, RANK, ROW_NUMBER	21, 24
'running total / compare with previous'	window function SUM() OVER, LAG()	21
'if-else in a column'	CASE WHEN	20
'duplicates'	GROUP BY col HAVING COUNT(*) > 1	23
'all or nothing / money transfer'	transaction + ROLLBACK	32
'two users editing together'	locks, isolation level	34-35
'query is slow'	EXPLAIN, add an INDEX	37-39
'repeated data, update bugs'	normalise to 3NF	26-28
'many-to-many'	junction table	6
'huge data, flexible schema'	NoSQL / sharding	43-44
'user input in a query'	parameterised query	41

Method

Write the ***question in plain English***, then underline the keywords. Each keyword maps to one SQL clause.

PRACTICE

Interview Q&A: Rapid Fire

Question	Crisp answer
DBMS vs RDBMS?	RDBMS stores data in related tables with keys; DBMS is the general idea.
Primary vs unique key?	PK: unique + not null, one per table. Unique: allows one NULL, many per table.
DELETE vs TRUNCATE vs DROP?	remove rows (filterable) / empty table fast / remove table itself.
WHERE vs HAVING?	WHERE filters rows before grouping; HAVING filters groups after.
INNER vs LEFT JOIN?	INNER keeps matches only; LEFT keeps all left rows.
What is normalization?	Organising tables to remove redundancy and anomalies (1NF → 3NF/BCNF).
What is ACID?	Atomicity, Consistency, Isolation, Durability.
What is an index?	A B+ tree structure that makes searches O(log n) at the cost of slower writes.
Clustered vs non-clustered?	Clustered orders the table rows (one per table); non-clustered is a separate lookup.
What is a deadlock?	Two transactions wait on each other's locks forever; DB aborts one.
View vs materialized view?	View = saved query; materialized view = stored result.
UNION vs UNION ALL?	UNION removes duplicates; UNION ALL keeps them (faster).
What is a foreign key?	Column that references another table's primary key.
SQL vs NoSQL?	Fixed schema + joins + ACID vs flexible schema + horizontal scaling.
What is SQL injection?	Malicious input altering a query; stop it with prepared statements.

Winning formula

Answer pattern: definition (1 line) → small example (1 line) → why it matters (1 line). Always give an example!

30-Day Plan & Common Mistakes

PRACTICE

Spend **1 hour a day**: 20 min reading these notes, 40 min writing queries in SQLite or MySQL.

Days	Topic	Do this
1–3	DBMS basics, ER, keys	draw 3 ER diagrams (library, shop, hospital)
4–6	SQL basics, filters	20 SELECT/WHERE questions
7–10	GROUP BY, HAVING, joins	solve 5 join problems daily
11–14	Subqueries, CASE, window	LeetCode SQL 50: first 25
15–18	Normalization	normalise 5 tables to 3NF; find keys
19–22	Transactions, locks	explain ACID with an example, aloud
23–25	Indexing, EXPLAIN	index a 100k-row table, compare time
26–27	NoSQL, CAP, security	build a tiny Python + SQLite app
28–29	Mock questions	15 rapid-fire answers + 10 SQL problems
30	Mock interview	45 min, speak while you type SQL

Common mistakes

Mistake	Fix
UPDATE / DELETE without WHERE	test with SELECT first; use a transaction
`= NULL` instead of IS NULL	always IS NULL / IS NOT NULL
Non-aggregated column not in GROUP BY	add it to GROUP BY or aggregate it
Using NOT IN with NULLs	use NOT EXISTS
Missing index on JOIN column	index foreign keys
String-building SQL in code	parameterised queries

Study trick
Handwrite the key definitions (ACID, normal forms, joins). Writing locks them into memory better than reading.

PRACTICE

References & Further Learning

These notes are in my own words with **original hand-drawn diagrams**. Every SQL and Python example was run and checked. These resources guided the topic order and are great for going deeper.

Roadmaps and syllabus

- **Scaler:** DBMS roadmap 2026: https://www.scaler.com/blog/dbms-roadmap-2026-complete-learning-path-for-beginners-to-advanced/
- **Scaler:** DBMS syllabus overview: https://www.scaler.com/blog/?p=12648
- **Database Star:** SQL roadmap: https://www.databasestar.com/sql-roadmap/

Learn and practise SQL

- **SQLBolt:** interactive lessons: https://sqlbolt.com/
- **SQLZoo:** https://sqlzoo.net/
- **W3Schools SQL:** https://www.w3schools.com/sql/
- **LeetCode:** SQL 50 study plan: https://leetcode.com/studyplan/top-sql-50/
- **HackerRank:** https://www.hackerrank.com/domains/sql

Read and go deeper

- **GeeksforGeeks:** DBMS tutorial: https://www.geeksforgeeks.org/dbms/
- **Use The Index, Luke:** how indexes work: https://use-the-index-luke.com/
- **SQLite docs:** https://www.sqlite.org/docs.html
- **Book:** *Database System Concepts* by Silberschatz, Korth, Sudarshan
- **Tool:** draw schemas free at https://dbdiagram.io/

More from us
Visit the **PrepPath E-books Store** website to explore more e-books for your learning journey. Shop now!